

Documentación ALU extendida

Para implementar las funciones extendidas fue necesario entender el concepto general del flujo de la ALU estándar, principalmente el hecho de que todas las operaciones se realizan en paralelo y al final se toma la decisión de qué salida es la que corresponde según la operación que se pedía, no al contrario, como se suele entender: identificar la operación que se debe hacer y realizar únicamente dicha operación.

Por este mismo motivo es que las operaciones extendidas se realizan siguiendo el mismo flujo de las operaciones estándar, luego del procesamiento inicial de x y y , de modo que se realizan a la par.

Para implementar la función $XOR16$ solicitada para la ALU extendida fue necesario realizar primero el XOR en la misma plataforma (dentro del proyecto 1) ya que la lógica era realmente similar, solo que debía extenderse a 16 bits. Esto significa que no realiza una única comparación entre las entradas a y b , ambas de un bit, que devuelve una salida out , también de un bit. Sino que aplica un xor entre el bit $a[i]$ y $b[i]$, y así sucesivamente a lo largo de los 16 bits de ambas entradas, realizando dicho procedimiento un total de 16 veces para generar una salida de 16 bits.

Un ejemplo de dicho funcionamiento es el siguiente:

Dadas las entradas X y Y de 16 bits se obtiene el resultado R :

Recordando la tabla de verdad del Xor $[0 | 0 = 0; 0 | 1 = 1; 1 | 0 = 1; 1 | 1 = 0]$

X: 0 1 0 0 1 0 1 1 1 0 1 0 0 1 0 1

Y: 1 1 0 0 0 0 1 0 1 1 0 0 1 0 0 1

R: 1 0 0 0 1 0 0 1 0 1 1 0 1 1 0 0

Como se observa en el test manual:

Chip ALU		Eval	Reset	Clock: 0
Input pins				
x	0 1 0 0 1 0 1 1 1 0 1 0 0 1 0 1	dec		
y	1 1 0 0 0 0 1 0 1 1 0 0 1 0 0 1	dec		
a	0			
zx	0			
nx	1			
zy	0			
ny	1			
f	1			
no	0			
Output pins				
out	1 0 0 0 1 0 0 1 0 1 1 0 1 1 0 0	dec		
zr	0			
ng	1			

La lógica detrás de la función *NAND* consiste en un *And16*, que aplica la función *And* entre el par de bits $a[i]$ y $b[i]$ de ambas entradas de 16 bits ($a[16]$, $b[16]$). Dado el resultado de 16 bits del *And16*, este ingresa a un *Not16*, donde se le aplica el Not a cada bit y se devuelve una salida de 16 bits que corresponde al *NAND* entre las entradas a y b .

Un ejemplo de esta función es:

Dadas las entradas X y Y de 16 bits se obtiene el resultado R :

Recordando la tabla de verdad del *And* [$0 \mid 0 = 0$; $0 \mid 1 = 0$; $1 \mid 0 = 0$; $1 \mid 1 = 1$] y la del *not* [$0 = 1$; $1 = 0$]

X : 1 1 0 0 0 1 0 0 1 1 1 0 0 0 1 0

Y : 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0 0

R : 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1

Como se observa en el test manual:

The screenshot shows a digital logic simulator interface. At the top, there are buttons for 'Eval', 'Reset', and 'Clock: 0'. Below these, there are two 16-bit input registers labeled 'X' and 'Y'. The 'X' register contains the value 1 1 0 0 0 1 0 0 1 1 1 0 0 0 1 0, and the 'Y' register contains 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0 0. Below the inputs, there are several status indicators: 'ns', 'zx', 'nx', 'zy', 'ny', 'f', 'no', 'zr', and 'ng'. The 'no' indicator is set to 1, and the 'ng' indicator is also set to 1. At the bottom, there is a 16-bit output register labeled 'out' which contains the value 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1, matching the expected result R. The output is displayed in a blue bar with a 'dec' button next to it.

Cabe aclarar que en la implementación de la función *NAND* en la alu extendida no se incluyó el *Not16* posterior al *And16*, únicamente se aplica la función *And16* porque el código de la función *NAND* al ser 000001, implica que $no = 1$ y al final del código se

encuentra la estructura de la alu estándar donde, dado $no = 1$ entonces se niega la salida, por lo que no es necesario agregar el *Not16* luego del *And16*, ya que sería una doble negación.

De manera similar, el funcionamiento del *NOR* consiste en la aplicación del *Or* a cada par de bits $a[i]$ y $b[i]$ de ambas entradas de 16 bits, que se realiza a través de un *Or16*, y nuevamente la negación de la salida a través de un *Not16*, que funciona de la misma manera en que se detalló anteriormente.

Un ejemplo de esta función es:

Dadas las entradas X y Y de 16 bits se obtiene el resultado R :

Recordando la tabla de verdad del *Or* [$0 \mid 0 = 0$; $0 \mid 1 = 1$; $1 \mid 0 = 1$; $1 \mid 1 = 1$] y la del *not* [$0 = 1$; $1 = 0$]

X: 1 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1

Y: 0 0 1 0 1 0 0 0 0 0 1 1 0 1 0 0

R: 0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0

Como se observa en el test manual:

The screenshot shows a digital logic simulator interface with a dark background. At the top, there are buttons for 'Eval', 'Reset', and 'Clock: 0'. Below these, there are two sections: 'ns' (inputs) and 'pins' (outputs). In the 'ns' section, there are two 16-bit input registers labeled 'x' and 'y'. The 'x' register contains the value 1 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1, and the 'y' register contains 0 0 1 0 1 0 0 0 0 0 1 1 0 1 0 0. Below these, there are several single-bit output indicators: 'a' (0), 'zx' (1), 'nx' (1), 'zy' (0), 'ny' (1), 'f' (0), and 'no' (0). In the 'pins' section, there are three outputs: 'out' (a 16-bit register containing 0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0), 'zr' (0), and 'ng' (0). Each register has a 'dec' button next to it.

Component	Value
Input x	1 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1
Input y	0 0 1 0 1 0 0 0 0 0 1 1 0 1 0 0
Output a	0
Output zx	1
Output nx	1
Output zy	0
Output ny	1
Output f	0
Output no	0
Output out	0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0
Output zr	0
Output ng	0

La igualdad (eq) fue una estructura que requirió mucho más circuitos y análisis debido a la complejidad, el flujo de la implementación pensado fue:

1. Era necesario emplear una estructura que pudiese “detectar” si dos bits son iguales o diferentes al realizar la comparación entre el par de bits $a[i]$ y $b[i]$ de ambas entradas de 16 bits. La idea de “detectar” se logra utilizando un *Xor16* que indica si hay coincidencia con un 0 o si hay diferencia con un 1, entonces el resultado de 16 bits va a contar por lo menos un 1 si las dos entradas son diferentes y únicamente ceros si son iguales.
2. La segunda estructura necesaria es una que pueda identificar si hay alguna diferencia, es decir, que pueda evaluar si hay algún 1 en el resultado del *Xor16*, para lo cual se emplean dos *Or8Way* que generaran un resultado de un solo bit a partir de una entrada de 8 bits, es decir, el resultado del *Xor16* se parte en dos para ingresar al primer *Or8Way* los primeros 8 bits (los menos significativos 0..7) y al segundo los últimos 8 (los más significativos 8..15).
El resultado final serán dos bits diferentes, cada uno generado por un *Or8Way*, que serán ambas 0 si no hubo ninguna diferencia en ninguna de las dos mitades, o podrían ser ambas 1 si había al menos una diferencia en ambas mitades.
3. Es necesario consolidar ambas salidas en una sola, para lo que se emplea un *Or* sencillo entre ambos bits resultantes de los dos *Or8Way* y así se llega a una variable que será 1 si los números son diferentes (hubo diferencia en por lo menos un bit) o 0 si los números son iguales.
4. Posteriormente se le aplica un *Not* simple al bit resultante del punto anterior para que se cumpla con las indicaciones solicitadas: el resultado es 1 (en binario) si son iguales, y 0 si son diferentes, y hasta el momento las variables indicaban al contrario.
5. Finalmente, para poder elegir y seleccionar la salida del *Eq* se emplea un *Mux* que tiene como selector la variable generada en la negación anterior, en la entrada a la variable *false*, que se traduce en el número 0 extendido a 16 bits (16 ceros) y en la variable *b* la variable que llamamos ‘uno’, que se genera gracias a un *Inc16* que se realizó antes del *Mux* y tenía por entrada un *false*. Se debe emplear un *Inc16* porque si se utilizara un *true* en el *Mux*, como se empleo el *false*, el resultado no sería 15 bits en 0 y el primero un 1 sino 16 unos ya que el *mux* extiende la variable. Por otro lado, el *Inc16* solo suma uno al último número, que es lo que se necesitaba como resultado en caso de que *a* y *b* fueran iguales = 0000000000000001.

La lógica entonces es: si el selector es 1 se elige la salida *b*, que significa que fueron iguales los dos números, pero si el selector es 0 elige la variable *a* que es el *false* e indica que son diferentes.

Un ejemplo de esta función es:

Dadas las entradas X y Y de 16 bits se obtiene el resultado R :

Recordando la tabla de verdad del xor [$0 \mid 0 = 0$; $0 \mid 1 = 1$; $1 \mid 0 = 1$; $1 \mid 1 = 0$], del or [$0 \mid 0 = 0$; $0 \mid 1 = 1$; $1 \mid 0 = 1$; $1 \mid 1 = 1$] y la del not [$0 = 1$; $1 = 0$]

Siendo $y = x$

X : 1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 0

Y : 1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 0

R : 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

Como se observa en el test manual:

The screenshot shows a digital logic simulator interface. At the top, there are buttons for 'Eval', 'Reset', and 'Clock: 0'. Below this, there are two input sections labeled 'ns' and 'pins'. The 'ns' section has two inputs, 'x' and 'y', both set to the binary value '1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 0'. The 'pins' section has three outputs: 'out' is set to '0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1', 'zr' is set to '0', and 'ng' is set to '0'. Each input and output field has a 'dec' button next to it.

Input	Value
x	1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 0
y	1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 0

Output	Value
out	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
zr	0
ng	0

Siendo $x \neq y$

X : 1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 1

Y : 1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0

R : 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Como se observa en el test manual:

The screenshot shows a digital logic simulator interface with a dark background. At the top, there are buttons for 'Eval', 'Reset', and 'Clock: 0'. Below this, there are two main sections: 'Inputs' and 'Outputs'. In the 'Inputs' section, there are eight rows, each with a label and a 16-bit value in a blue box, followed by a 'dec' button. The inputs are: x (1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 1), y (1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0), a (0), zx (1), nx (0), zy (1), ny (0), f (0), and no (0). In the 'Outputs' section, there are three rows: out (0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0), zr (1), and ng (0). Each output also has a 'dec' button.

Input	Value
x	1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 1
y	1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0
a	0
zx	1
nx	0
zy	1
ny	0
f	0
no	0

Output	Value
out	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
zr	1
ng	0

Por último, la estructura ABS es implementar circuitos que permitan realizar el complemento a 2, ya que identificar si el número es negativo o positivo se puede realizar observando el bit más significativo.

El complemento a 2 se realiza invirtiendo la entrada, en este caso lo realizamos con un Not16, teniendo en cuenta que en este caso solo nos preocupamos por la entrada x y la y se ignora. Luego de que el número ha sido invertido bit a bit, se debe incrementar en uno el resultado, es decir, se emplea un Inc16 sobre el resultado del Not16.

Posteriormente se emplea un Mux para decidir si el resultado del valor absoluto es el número sin modificaciones, es decir, x era positivo; o el número transformado, en cuyo caso x era negativo. El Mux utiliza como selector x[15] el bit más significativo. Si este bit es 0, entonces se elige la variable a que devuelve a x, pero si x[15] es 1, entonces se elige para salir a la variable b, que contiene el resultado de todo el proceso de complemento a 2.

Un ejemplo de esta función es:

Dada la entrada X de 16 bits se obtiene el resultado R:

Recordando la tabla de verdad del *not* [0 = 1; 1 = 0]

Dado $x < 0$

X: 1 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0

Y: 1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0 (puede ser cualquier cosa, no afecta el resultado)

R: 0 1 0 1 0 1 1 1 1 0 1 1 1 0 0 0

Como se observa en el test manual:

The screenshot shows a digital logic simulator interface with a dark background. At the top, there are buttons for 'Eval', 'Reset', and 'Clock: 0'. Below these, there are two main sections: 'Inputs' and 'Outputs'. In the 'Inputs' section, there are eight rows, each with a label and a value in a blue box: 'x' (1 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0), 'y' (1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0), 'a' (0), 'zx' (1), 'nx' (0), 'zy' (0), 'ny' (0), and 'f' (1). Below these is a 'pins' section with three rows: 'out' (0 1 0 1 0 1 1 1 1 0 1 1 1 0 0 0), 'zr' (0), and 'ng' (0). Each input and output value is displayed in a blue box, and each row has a 'dec' button to the right.

Label	Value
x	1 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0
y	1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0
a	0
zx	1
nx	0
zy	0
ny	0
f	1
no	0
out	0 1 0 1 0 1 1 1 1 0 1 1 1 0 0 0
zr	0
ng	0

Dado $x > 0$

X: 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0

Y: 1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0 (puede ser cualquier cosa, no afecta el resultado)

R: 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0

Como se observa en el test manual:

U

Eval

Reset

Clock: 0

ns

x

0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0

dec

y

1 0 1 0 0 1 0 1 0 1 0 0 1 0 1 0

dec

a

0

zx

1

nx

0

zy

0

ny

0

f

1

no

0

pins

out

0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0

dec

zr

0

ng

0

Luego de comprender como se implementaron cada una de las operaciones de la Alu extendida es también necesario definir como se genera la salida, ya que, como se mencionó al inicio, la Alu realizó ya todas las operaciones, tanto de la extendida como de la estándar, y debe decidir cuál de todas las respuestas es la respuesta final según la operación solicitada.

Para esto se requiere implementar una secuencia de fórmulas que se activan (devuelven una variable de un bit cuyo valor es 1) cuando detectan que el código de la operación ha sido ingresado (cuando detectan la combinación de valores de las entradas zx, zy, nx, ny, f y no específica del código de esa operación). Estas fórmulas se construyen realizando una tabla de verdad con los códigos y las entradas, de modo que donde la entrada zx debía ser = 0, entonces la operación implementada en el código es un Not sobre esa entrada zx y si fuese 1 entonces no se necesita negar. Se va realizando este procedimiento por cada entrada y posteriormente se juntan con And el resultado de todas.

Realicemos el ejemplo con el código de *XOR*

Operación	zx	nx	zy	ny	f	no
Xor	0	1	0	1	1	0

Dados esos valores las operaciones a implementar serían:

1. Un not sobre zx, zy y no
2. Un And sobre notzx (zx ya negado) y nx
3. Luego dicho resultado unirlo a un And que incluya también el de not zy (con zy ya negado)
4. Luego se genera un And entre la salida anterior y el ny sin negar
5. Luego dicho salida pasa por un And con f sin negar
6. Luego se concatena ese resultado por medio de otro and con el notno resultante del punto 1.

Luego de toda esta estructura (que se repitió para las 5 operaciones según su propia combinación) es que la variable resultante será un 1 si esa fórmula si es la ingresada o un 0 si no, y pasa a una siguiente etapa esa variable tan importante.

Ya obtuvimos un indicador que nos sirve para buscar como identificar cuál operación de las 5 nuevas operaciones va a sacar como resultado la Alu extendda. Y para esto emplea diferentes Mux a lo largo de todo el procesamiento de las operaciones. En total se emplean un Mux por cada una de las operaciones extendidas para este propósito (además de las empleadas para realizar las operaciones).

El Mux recibe como selector el resultado final de la fórmula de esa operación (que indica si se activó o no) y de ser un 1 (se activó), selecciona la salida de la operación (que se encuentra en la variable b), ya que estos Mux se ubican cada uno debajo de todo el cálculo de su proceso. Y si no se activa deja salir la variable a, que es la salida resultante del Mux selector de operación anterior, es como decir: “no activaron mi código entonces yo no muestro mi resultado pero dejo pasar el del anterior porque hasta el momento parece que ese ha sido llamado” (excepto el primero que de entrada a tiene la variable false)

Por último se realiza un conjunto de 4 Or entre todas estas variables resultantes de las fórmulas de modo que se llegue a una sola variable de un bit que simbolice si se ingresó un código (independiente de cuál) de la Alu extendida o no, es decir, es una variable que indica si la Alu extendida tiene la salida final o no.

Y es esa variable la que se ingresa en la misma estructura de la Alu estándar que calcula las flags zr y ng, de modo que la entrada del Not16 que hace parte del procesamiento de “Si no=1 entonces negar salida” se cambia del valor que tenía en la Alu estándar por esta variable final de si en la Alu extendida se registró un código de una operación de ellos o no. Y así ya se realiza la elección normal que ya se realizaba con la misma estructura en la Alu estándar, ya que la única modificación fue esa variable de entrada.

Y de esta manera funciono y se implementó todo el proceso de las funciones extendidas de la Alu y la toma de decisiones dentro de ella.